SPRINT 6 · DATOS LISTOS PARA MODELAR · S06.2

Limpieza y preprocesado

El cliente nos envía un segundo fichero, reservas_sucio.csv, exportado a mano desde varios sistemas. Limpiarlo es entre el 60 y el 80 % del trabajo.



Nunca limpies a ciegas

```
crudo = pd.read_csv(dato("reservas_sucio.csv"))   # el original no se toca
crudo.info(); crudo.isna().sum(); crudo.describe()
```

1. Mide (diagnóstico).
2. Decide el tratamiento y anota por qué.
3. Escribe el código.
4. Comprueba que mejoró.

Cada paso crea una copia nueva (d1, d2, d3…): puedes ver cuánto cambia cada decisión.

Los defectos del fichero sucio (cuaderno S06_02)

Defecto	Cómo se arregla	Ejercicio
Filas duplicadas	drop_duplicates() y cuenta n_duplicados	1
Precios con coma decimal («84,36»)	str.replace(",", ".") y astype(float)	2
Fechas en dos formatos	pd.to_datetime(..., format=..., errors="coerce") por grupo	3
«Directo», «directo », «DIRECTO»	strip().lower().replace(" ", "_")	4
Valores imposibles y outliers	reglas de negocio y NaN	5

Fechas, imposibles y outliers

- No adivines el formato de fecha: 2024-11-19 es ISO y 19/11/2024 es día/mes/año; lee cada grupo con su formato.
- Dos clases de valores extraños: Imposibles (precio negativo, 0 noches): error seguro → NaN. Outliers (precio muy alto pero posible): se investigan, no se borran sin más.
- Consistencia entre columnas: fecha_llegada no puede ser anterior a fecha_reserva.

```
iso = pd.to_datetime(d2["fecha_reserva"], format="%Y-%m-%d", errors="coerce")
esp = pd.to_datetime(d2["fecha_reserva"], format="%d/%m/%Y", errors="coerce")
d3["fecha_reserva"] = iso.fillna(esp)
d5.loc[~d5["precio_noche"].between(1, 1000), "precio_noche"] = np.nan
```

Nulos: ¿qué hacemos con ellos?

Antes de imputar, pregunta por qué falta el dato.

Falta al azar

Imputar con mediana (numéricas) o moda (categóricas) es razonable.

Falta por algo

Un pais_origen vacío puede ser información: crear la categoría «desconocido».

Faltan demasiados

Si una columna está vacía en su mayoría, mejor descartarla.

Mini-reto: empaqueta todo en tu función limpiar_reservas(df): reproducible, nada de arreglar el CSV a mano.

Separar entrenamiento y prueba (cuaderno S06_03)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEMILLA)    # Ejercicio 1
```

- El 20 % de los datos no se toca hasta el final: el modelo nunca lo ve al aprender.
- stratify=y mantiene la misma proporción de canceladas en ambos trozos.
- random_state hace la partición repetible.

Escalado y codificación

```
esc_minmax = MinMaxScaler().fit(X_train[["precio_noche", "ninos"]])    # Ejercicio 2
X_train_esc = esc_minmax.transform(X_train[["precio_noche", "ninos"]])

OrdinalEncoder(categories=[["sin_deposito", "parcial", "total"]])     # Ejercicio 3
OneHotEncoder(handle_unknown="ignore")       # canal, regimen, tarifa...
```

- precio_noche va de 50 a 250 y ninos de 0 a 3: los modelos basados en distancias los necesitan en la misma escala.
- Ordinal si hay orden (deposito: 0, 1, 2); one-hot si no lo hay (canal).
- fit solo con entrenamiento, transform con entrenamiento y prueba.

ColumnTransformer y Pipeline: la receta completa

```
preprocesado = ColumnTransformer([                  # Ejercicio 4
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("esc", StandardScaler())]), columnas_num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("oh", OneHotEncoder(handle_unknown="ignore"))]), columnas_cat),
])
modelo = Pipeline([("prep", preprocesado),          # Ejercicio 5
                   ("clf", LogisticRegression(max_iter=1000))])
```

Cada tipo de columna necesita su tratamiento, y todo queda en un solo objeto que se ajusta y se aplica igual siempre.

Fuga de datos en el preprocesado

Hay fuga cuando el modelo «ve» información del conjunto de prueba durante el aprendizaje.

- Error típico: escalar todo el dataset y luego partir. La media y la desviación ya incluyen la prueba.
- Solución: meter el escalado dentro del Pipeline; así se aprende solo con el trozo de entrenamiento (también dentro de la validación cruzada).
- Un vistazo a PCA: reducir columnas conservando la varianza (lo veremos a fondo en el Sprint 9).
- Mini-reto: comprueba que tu pipeline es reproducible (mismas predicciones con la misma semilla).

S06 · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S06_02_limpieza

55 min

Limpieza: duplicados, tipos, fechas, categorías, imposibles y nulos (los ejercicios marcados «en clase»)

sprints/S06/2_Practica/S06_02_limpieza.ipynb

CUADERNO

S06_03_preprocesado_pipeline

40 min

Escalado, codificación, ColumnTransformer y Pipeline (los ejercicios marcados «en clase»)

sprints/S06/2_Practica/S06_03_preprocesado_pipeline.ipynb

RETO

Reto del sprint

30 min

Función de limpieza, partición train/test, *pipeline* y *baseline* (pasos 3–4)

sprints/S06/3_Reto/2_Ficha_completa/Reto_Sprint_06.md